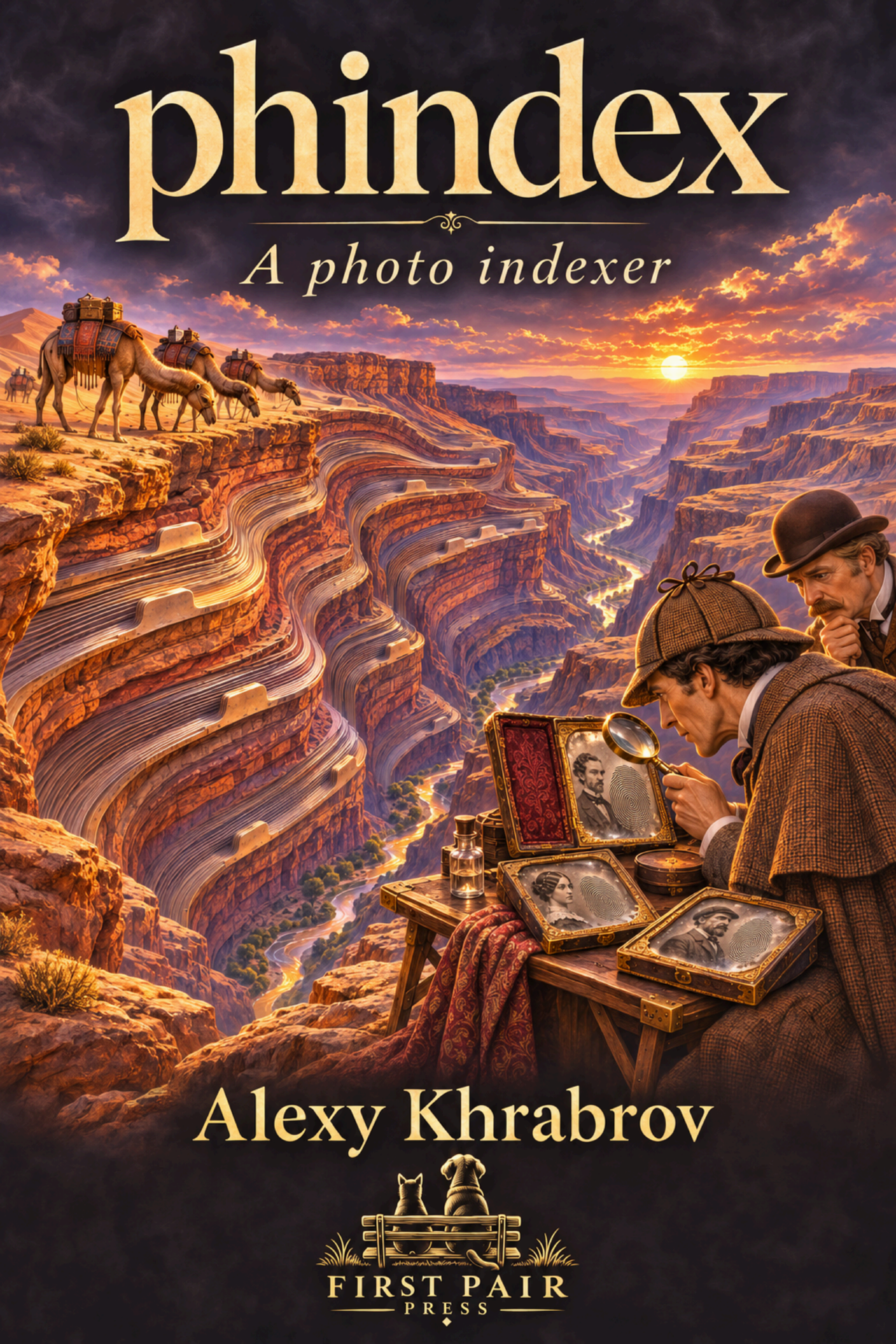
phindex

A photo indexer



Alexy Khrabrov

FIRST PAIR
PRESS

phindex

An index of photo identities that is cheap to rerun

Alexy Khrabrov

2026-11-02

Version 0.1.0-10d23761
Source commit 10d23761a37c0f520ad75b030da084680e706d71

First Pair Press
firstpair.press

Contents

About this manual 4
What phindex is 5
The identity columns 5
Three states, and never a match on failure 6
Where it fits 6
What it is not 7
Building and running 8
phindex (Rust) 8
phindex-ox (stock OCaml and OxCaml) 8
The command line 8
What it prints 9
The schema 10
files 10
metadata 10
dirs 11
runs 11
Incremental behaviour 12
What a rerun reads 12
The revision stamp 12
The quick walk 12
In-place rewrites and `--full-stat` 13
`--only` 13
Moves and renames 13
Gone 14
Commits, interruptions and resuming 14
A file that changes during the run 14
Two implementations, one index 15
phindex: Rust 15
phindex-ox: OCaml, stock and OxCaml 15
The comparison 15
Performance and tuning on hard disks 17
Where the time goes 17
When workers help 17
When the disks limit 17
Small files lower MB/s, not files per second 17
Memory 18
Queries 19
Paths under a root 19
Containment between two roots 19
Metadata-only differences 19
Lightroom catalog matching 20
Housekeeping 20
Testing 21
Licences 22

Reference 23

About this manual

This manual is by Alexy Khrabrov, with Claude as co-author. The ideas and decisions are Alexy's; Claude wrote the text and the code with him.

It describes phindex 0.1.0 as of 2026-11-02: the Rust binary `phindex` and its OCaml counterpart `phindex-ox`, which runs under stock OCaml 5 and under OxCaml. phindex is licensed under the MIT License; the binaries link code under other licences, as the chapter on licences explains. Its source is at github.com/firstpair/phindex.

Every example in this manual uses synthetic files or generic paths such as `/archive` and `/backup`. Nothing here depends on a particular photo library, disk or catalog.

What phindex is

phindex builds, and keeps up to date, an index of every regular file under a directory tree, in one SQLite file. The tree might be a photo archive, a backup volume or a disk that came back from a trip. For each file the index records where it is, its size, mtime, inode and device, and a set of **identities** computed by phidentify: the whole-file SHA-256, the DNG encoded raw blocks, the Lightroom Classic original fingerprint, the metadata record digest, and the tiffgraph grammar with the preserved digest. All of them come from one identification of the file: phidentify reads it into memory once and takes every identity it can from those bytes, and checks that the file did not change while it worked.

The index exists to answer archive questions without reading the archive again. Which of these files have a byte-identical copy on the backup? Which DNGs differ from their copy only in their XMP packet? Which file on disk is the original a Lightroom catalog entry recorded? Each of those is an SQL query against the index, and each takes seconds, where reading a multi-terabyte tree takes days.

So the index has to be cheap to keep current. Most of this manual is about that: a rerun reads only files that are new, changed or unfinished, finds changed directories from their mtimes without listing the rest, carries identities across moves and renames, and resumes after an interruption where it stopped.

The identity columns

Every identity has its own column in the `files` table, and every identity that can fail has a matching `_err` column for the reason.

Column	Identity	Attempted for	Equal values mean
`sha256`	whole-file SHA-256	every file	every byte is equal
`dng_blocks`, `dng_blocks_err`	DNG encoded raw blocks	files with a TIFF header	same encoded primary raw data; metadata may differ
`lr_digest`, `lr_err`	Lightroom Classic original fingerprint	files that parse as DNGs	Lightroom Classic's matcher would pair them
`md_digest`, `md_err`	exiftool-rs metadata record, file location left out	every file	same metadata record from the same engine
`dngsem_profile`, `dngsem_err`	strictest tiffgraph grammar that accepts the file	every file	the same grammar applies
`preserved`, `preserved_err`	Semantic Unison's `SDE/0-preserved` digest	files a grammar accepted	every byte outside the XMP packet is equal, at the same size and layout
`ids_rev`	the revision stamp of the identity definitions	every identified file	the identities were computed by the same definitions

phidentify's manual defines each identity exactly: what goes into it, what equality proves and what it does not, and what it costs. This manual does not repeat those definitions. In short:

- **`sha256`** is the only identity that proves two files are the same bytes. Files over 1 GiB get a streamed SHA-256 and nothing else that needs the bytes in memory.
- **`dng_blocks`** hashes the encoded raw image data of a DNG and its encoding tags, without offsets, XMP, previews or maker notes. A keyword edit cannot change it; a flipped raw byte always does.

- **`lr_digest`** is phidentify's reimplementation of the fingerprint Lightroom Classic keeps for every original, in upper-case hex as Classic stores it. It is computed only for the DNG strata phidentify supports; for anything else it is an error, never a guess.
- **`md_digest`** hashes the ExifTool-shaped record that dng-compare's exiftool-rs engine writes, without the members that say where the file sits (its name, directory and file dates). The engine's own version stamp stays in the hash, so records from different engines never compare equal.
- **`dngsem_profile`** names the strictest of the grammars `dng-xmp/0`, `dng-xmp/1` and `dng-xmp/2` under which tiffgraph found the primary XMP packet and checked that nothing else the file declares lies inside it.
- **`preserved`** hashes every byte outside that packet, with the file size and the packet's position bound in. Two files with equal `preserved` values under the same `dngsem_profile` can differ only inside the packet.

Three states, and never a match on failure

Each identity column is in one of three states:

State	Value column	`_err` column	Example
computed	the value	NULL	a supported DNG's `lr_digest`
failed or refused	NULL	the reason	`dngsem_err` = `read outside the bytes` for a file too short to be a TIFF
not attempted	NULL	NULL	`dng_blocks` for a text file, which has no TIFF header

A NULL value is never a match. SQL already treats it that way, since `NULL = NULL` is not true, and the queries in this manual rely on it: two files that both failed an identity have not agreed on anything. The reason is kept so that you can tell an identity that does not apply from one that broke, and count either.

A file whose identification failed as a whole, because it could not be read or because it changed while it was being read, keeps `sha256` NULL and every identity column NULL. Its reason is printed as an `ERROR` line on stderr (the first 50 of a run are printed) and counted in the run's errors. It is read again on the next run.

Where it fits

phindex is the part of the photo-archive work that remembers. The other parts compute, compare and transfer:

- **phidentify** computes the identities, as a Rust library with a C ABI and an OCaml package that builds under stock OCaml and under OxCaml. phindex never reimplements an identity; it calls phidentify and stores what comes back. phidentify's manual is on the First Pair shelf.
- **tiffgraph** is a bounded TIFF/DNG graph walker, in OCaml (stock OCaml and OxCaml) and in Rust. Under a fixed grammar it locates a DNG's editable XMP packet and refuses anything outside the grammar with a reason. The `dngsem_profile` and `preserved` columns come from its walk: the Rust crate inside phidentify's Rust core for `phindex`, the OCaml library inside phidentify's OCaml package for `phindex-ox`. Its manual is on the First Pair shelf.
- **Semantic Unison** is a fork of the Unison file synchronizer that sends DNG metadata edits without resending image bytes. Its peers negotiate which tiffgraph grammar they trust, and it defines the `SDE/0-preserved` digest that proves a metadata transfer left every other byte alone. The `preserved` column is that digest, so an index built by phindex speaks the same identity as Unison. Its manual is on the First Pair shelf.

- **dng-compare** compares DNG archives: it pairs files across trees, compares raw samples, decodes when encodings differ, and keeps metadata archives. phidentify's raw-block identity is dng-compare's block comparison turned into a digest, and its metadata identity uses dng-compare's exiftool-rs engine. Its manual is on the First Pair shelf.

What it is not

- **Not a writer.** phindex only reads the indexed tree. It writes nothing but its SQLite file.
- **Not a synchronizer or a deduplicator.** It tells you which files agree on which identity. What to copy, keep or delete is your decision, made from queries.
- **Not a catalog tool.** It can compare its `lr_digest` column with a Lightroom Classic catalog's stored fingerprints, read from a copy of the catalog. It never opens a live catalog and never writes one.
- **Not a server.** One indexer, `phindex` or `phindex-ox`, writes an index at a time. Readers can query the index while it runs, because it is in WAL mode.

Building and running

phindex (Rust)

The Rust toolchain is pinned in `rust/rust-toolchain.toml` to 1.99.0; rustup installs it on first use. phindex depends on phidentify from GitHub at a pinned commit, with the `lightroom`, `metadata` and `semantic` features.

```
git clone https://github.com/firstpair/phindex.git
cd phindex/rust
cargo build --release
cargo test --release
```

The result is one binary, `rust/target/release/phindex`. Index a tree:

```
phindex --db archive.sqlite --root /archive --workers 16
```

Run the same command again whenever the tree may have changed. To index a folder you have just added, without walking the rest:

```
phindex --db archive.sqlite --root /archive --only /archive/2026/new-card
```

phindex-ox (stock OCaml and OxCaml)

phindex-ox is built on phidentify's OCaml package, which binds phidentify's Rust library. It needs OCaml 5.1 or later, stock OCaml or OxCaml, dune 3.9 or later, the `sqlite3` opam package, zlib, and opam 2.1 or later for `--subpath`. Build phidentify's Rust library with the metadata identity first, then pin its OCaml package into the switch you want:

```
git clone https://github.com/firstpair/phidentify
(cd phidentify/rust && cargo build --release -p phidentify-ffi --features metadata)
PHIDENTIFY_FFI_LIBDIR=$PWD/phidentify/rust/target/release \
  opam pin add phidentify ./phidentify --subpath ocaml
cd phindex/ocaml && dune build
./_build/default/phindex_ox.exe --db archive.sqlite --root /archive --workers 16
```

The installed name is `phindex-ox`. The same source builds in a stock OCaml switch and in an OxCaml switch; phidentify's package compiles the domain pool that suits the switch, and the walk line reports which one is running (`domains-stock` under stock OCaml, `domains-oxcaml` under OxCaml). phindex-ox refuses to start when the linked phidentify lacks the `+lightroom` or `+metadata` identities its revision stamp promises.

phindex-ox's C stub reads the nanosecond mtime from macOS's `st_mtimespec`, so as written it builds on macOS. The Rust phindex uses only portable Unix metadata.

The command line

Both indexers take the same options; phindex-ox has one more.

Option	Default	Meaning
`--db FILE`	required	the SQLite index; created if missing, its schema migrated if older
`--root DIR`	required	the tree to index; canonicalized, so stored paths are absolute and free of symlinks
`--only SUBDIR`	none	walk only this folder, which must lie under `--root`
`--full-stat`	off	list every directory and stat every file, instead of the quick walk
`--workers N`	8	identification workers: rayon threads in phindex, OCaml domains in phindex-ox; at least 1
`--progress-secs S`	60	seconds between progress lines on stderr
`--batch B`	workers × 8	phindex-ox only: files handed to the domain pool at a time

Paths are stored as the canonical absolute path. On macOS, for example, a root given under `/tmp` is stored under `/private/tmp`. Write queries against the stored form.

What it prints

Everything goes to stderr. A run prints the walk, the plan, progress lines, and a summary:

```
walk: 3104 directories, 12 listed, 3092 unchanged and taken from the index, 1 s
walked 1612934 files under /archive, 412 to read (9.8 GB), 37 moved or renamed
(identities kept), 5 gone, 0 errors, 16 workers (phindex), identities: phidentify
0.1.0 +lightroom +metadata; tiffgraph dng-xmp/0,1,2 max_xmp=1MiB; SDE/0-preserved
+0h01m 288/412 files, 6.9/9.8 GB (70.4%), 115 MB/s, 0 errors, ETA 0h00m; raw blocks
263, Lightroom 251, metadata 288, preserved 259
done: read 412 files, 9.8 GB, 0 errors (0 stored SHA-256 mismatches); raw blocks 381,
Lightroom 362, metadata 412, preserved 376
```

The numbers are illustrative. The walk line says how many directories were listed and how many were taken from the index. The plan line says how many files must be read, how many were carried across a move, and how many vanished. The progress line counts, for the files read so far, how many got each identity. Its ETA assumes the remaining bytes go at the rate so far. In phindex the writer wakes every second, so progress lines arrive on time; in phindex-ox they are checked after each batch.

The schema

phindex creates four tables and migrates older ones by adding missing identity columns. Both indexers create the same schema.

files

One row per path ever seen under an indexed root.

Column	Type	Meaning
`path`	TEXT, primary key	canonical absolute path
`size`	INTEGER	size in bytes at the last walk that stat'ed it
`mtime_ns`	INTEGER	modification time in nanoseconds since the epoch
`ino`	INTEGER	inode number
`dev`	INTEGER	device number
`sha256`	TEXT	whole-file SHA-256; NULL until the file is identified
`hashed_unix`	INTEGER	when it was identified, in Unix seconds
`seen_unix`	INTEGER	when the row was created or last reset
`gone`	INTEGER	1 once the path has vanished; the row and its identities are kept
identity columns	TEXT	as in the first chapter, with `ids_rev`

Indexes cover `sha256`, `size`, `dng_blocks`, `lr_digest`, `md_digest` and `preserved`, so joins on any identity are fast.

A row is **complete** when `sha256` is not NULL and `ids_rev` equals the running indexer's revision stamp. Only complete rows are skipped on a rerun and only complete rows can pass their identities to a moved file.

metadata

One row per identified file that yielded a metadata record.

Column	Meaning
`path`	the file's path, as in `files`
`engine`	`exiftool-rs (via phidentify)`
`record_deflate`	the compacted record, raw-deflated

The record is the JSON object exiftool-rs wrote for the file, **compacted**: every string member longer than 1 KiB, such as a base64 thumbnail or a camera-profile table, is replaced by `long-sha256:<hex>:<length>`, the SHA-256 and byte length of the original string. The compacted JSON is then compressed with raw deflate, without a zlib header, as Rust's flate2 `DeflateEncoder` writes it; phindex-ox's C stub writes the same format. `md_digest` is always taken over the full record, never the compact one, so the stored record serves for comparison and display, and the digest for equality.

Because the records are kept, two files' metadata can be compared field by field later without reading either file. To read one back in Python:

```
import json, sqlite3, zlib
db = sqlite3.connect("file:archive.sqlite?mode=ro", uri=True)
(blob,) = db.execute("SELECT record_deflate FROM metadata WHERE path = ?",
                     ("/archive/2024/IMG_0001.dng",)).fetchone()
record = json.loads(zlib.decompress(blob, -15))
print(record["ExifTool:DngCompareEngine"], record["File:FileType"])
```

The record carries the engine's stamp, `ExifTool:DngCompareEngine`. Compare records only between equal stamps.

dirs

One row per directory ever visited: `path`, `ino`, `mtime_ns`, `seen_unix` and `gone`. The inode and mtime are taken before the directory is listed. This table is what makes the quick walk possible.

runs

One row per completed run: `root`, `started_unix`, `finished_unix`, `walked` (files in the walked scope), `hashed` (files identified), `bytes_hashed`, `gone` (files newly marked gone) and `errors`. Both indexers record the walked scope in `root`: the `--only` folder when one was given, otherwise `--root`. A run that was interrupted leaves no row.

Incremental behaviour

What a rerun reads

A file is read when any of these is true:

- its path is new to the index;
- its size, mtime or inode differs from its row;
- its row is not complete: it was never identified, its identification failed, the previous run was interrupted before reaching it, or its `ids_rev` differs from the running revision stamp.

Everything else is skipped without opening the file.

The revision stamp

Each identified row is stamped, in `ids_rev`, with the identity definitions that computed it:

```
phidentify 0.1.0 +lightroom +metadata; tiffgraph dng-xmp/0,1,2 max_xmp=1MiB; SDE/0-
preserved
```

The stamp names the phidentify identities and features, the tiffgraph grammars and their XMP limit, and the preserved-digest definition. It is the same constant in `phindex` and `phindex-ox`, so either can continue an index the other started.

The stamp changes only when an identity's definition changes: a new grammar, a different limit, a fix that changes some files' values. A release that leaves the definitions alone keeps the stamp, because a new stamp makes every row incomplete and the next run reads every file again. That full re-read is also a check: when it re-reads a file whose size, mtime and inode are unchanged and finds a different SHA-256 from the one stored, it prints `SHA256 CHANGED with unchanged size/mtime/inode`, counts the file as an error and in the summary's stored SHA-256 mismatches, and stores the new values. On an archive, such a line deserves attention: something changed the bytes without changing the metadata the file system keeps.

The quick walk

A directory's mtime changes whenever an entry is added to it, removed from it or renamed in it. phindex uses that. Every directory it visits is stored in `dirs` with its inode and mtime. On a rerun, for each directory:

1. phindex stats the directory.
2. If its inode and mtime equal the stored ones, the directory is **not listed**. Its files come from the index, with the size, mtime and inode stored for them, and its subdirectories come from `dirs`; each subdirectory is stat'ed in turn and goes through the same test.
3. Otherwise the directory is **listed**: every entry is stat'ed, new files and subdirectories are found, and vanished ones drop out.

So a rerun lists only the directories whose contents changed, such as the parent of a new folder and the folder itself, and stats one inode per unchanged directory. On a tree of 1.6 million files in 3,100 directories, the quick walk took 16 seconds. The first run of a tree, before `dirs` knows it, lists everything.

The directory's mtime is taken before it is listed. If an entry is added while the directory is being listed, the stored mtime is older than the directory's, so the next run lists it again.

Symbolic links are not followed, and only directories and regular files are indexed. A name that is not valid UTF-8 cannot be stored as SQLite TEXT; it is skipped and counted as an error.

In-place rewrites and `--full-stat`

The quick walk trusts a directory's mtime to say whether anything in it changed. That holds for every change to the directory's **entries**, but not for a change to a file's **contents**. A file rewritten in place under the same name, by an editor saving into it or a tool appending to it, changes its own mtime and size but not its directory's. The quick walk takes that file's row from the index and never sees the change.

`--full-stat` turns the quick walk off: every directory is listed and every file stat'ed. A file whose size, mtime or inode changed is then read again, exactly as on a first run. Use it after anything that may have rewritten files in place, such as a tool that writes metadata into DNGs, and periodically as a sweep. It costs one stat per file, not one read: on a large tree, minutes rather than seconds, and still nothing like reading the files.

Programs that save by writing a new file and renaming it over the old one change the directory's mtime, and the quick walk finds them.

`--only`

`--only SUBDIR` limits a run to one folder under `--root`, typically one you have just copied in. Only that folder is walked; only rows under it are considered for reading or for being marked gone; files elsewhere are left exactly as they are. The folder must lie strictly under the root, after both are canonicalized.

Moves are the exception, by design: a file that arrives in the `--only` folder from elsewhere in the index still inherits its identities, and its old path is marked gone even though it lies outside the folder, because the old path demonstrably no longer exists.

Moves and renames

Moving or renaming a file within one file system keeps its inode, size and mtime. phindex uses that to carry identities across a move without reading the file. For each path that is new to the walked scope, it looks for rows anywhere in the index with:

- the same device, inode, size and mtime;
- the same revision stamp, and a SHA-256, so they are complete;
- a path that is not in this walk and no longer exists on disk.

If there is one, the new path takes over its identities and its metadata record, and the old path is marked gone. Renaming a folder of three files is a run that reads nothing and reports three moved and three gone.

The last condition matters for hard links. Two live hard links share an inode, size and mtime, but both paths exist, so neither is taken for a move: each is read and stored separately.

A file moved twice can leave two old rows with the same device and inode, for example when it went from one folder to another between runs, was indexed there with `--only`, and was then moved again. Both old paths have vanished. The new path inherits from one of them, and every vanished old row that is not yet marked gone is marked gone, so the index never shows two live paths for one file that exists once.

A move across file systems is a copy and a delete. The new file has a new inode, so it is read; the old path is marked gone.

Gone

A path that was in the index under the walked scope and is no longer found by the walk is marked `gone = 1`. Its row and its identities stay, because they are evidence: they say what was there. Queries about the current tree add `gone = 0`. A path already marked gone is not counted again. Directories that vanish are marked gone in `dirs` the same way, and a directory that reappears is listed in full.

A gone path can come back. If it comes back unchanged, with the same inode, size and mtime it had when it vanished, as when a file is moved away and then moved back, it is marked present again (`gone = 0`) and keeps its identities; a complete row is not read again. If it comes back as a different file, with a new inode or a new size or mtime, its row is reset and read like any changed file.

Commits, interruptions and resuming

A run has two phases.

The plan is committed in one transaction before any file is read. It inserts rows for new paths and resets rows for changed ones (clearing every identity), marks vanished files and directories gone, records every visited directory's inode and mtime in `dirs`, and applies moves.

The reading commits every 500 results. phindex has one writer: the workers identify files in parallel and stream results over a bounded channel to the main thread, which writes them and commits after every 500 results, successes and failures alike. phindex-ox writes each batch's results in input order and commits after the batch that brings the count to 500 or more.

If a run is stopped, by Ctrl-C, a crash or a power cut, at most the uncommitted results are lost. The plan is already in the index: the files not yet read have rows with a NULL `sha256`, so they are incomplete. The next run's quick walk finds their directories unchanged, takes their rows from the index, sees them incomplete and reads them. Nothing already committed is read again. A run that finishes writes its row in `runs`.

One indexer writes an index at a time. Do not run phindex and phindex-ox, or two copies of either, against the same database at once.

A file that changes during the run

The plan records each file's size, mtime and inode. After phidentify has identified a file, the indexer stats it again. If it is no longer a regular file with the same size, mtime and inode, the result is discarded with the reason `changed since the walk`, counted as an error, and the row stays incomplete for the next run. Inside phidentify, a file that changes while it is being read is likewise an error, never a mixture of old and new identities.

Two implementations, one index

There are two indexers because the same work exercises two runtimes differently, and an index that both can write is a strong test of each. Both write the same tables with the same revision stamp, so either can continue the other's index, and they must agree row for row.

phindex: Rust

`phindex` is one binary. It calls phidentify's Rust core directly, with every identity requested, including the tiffgraph walk through tiffgraph's Rust crate, so the walk and every digest are taken from the same in-memory bytes.

Identification runs on a rayon pool of `--workers` threads. Each worker identifies a file, re-stats it, and sends the result over a bounded channel of 256 results to a single SQLite writer on the main thread. The writer compacts and deflates the metadata record, writes the row, commits every 500 results, and prints progress on time even when no result has arrived. The binary uses mimalloc as its global allocator.

phindex-ox: OCaml, stock and OxCaml

`phindex-ox` is written in OCaml on phidentify's OCaml package, which binds phidentify's Rust library through its C ABI and adds a domain pool and the tiffgraph walk in OCaml.

- **The pool depends on the switch.** Under **OxCaml**, phidentify's package uses a `Domain.Safe.spawn` pool. `Domain.Safe.spawn` accepts only portable closures, so the workers share nothing but an immutable array of paths and an atomic counter; the C externals carry the `@@ portable` modality. Under **stock OCaml 5**, the package uses a `Domain.spawn` pool with the same structure. dune picks the pool for the switch; phindex-ox's own source is the same in both.
- **The tiffgraph walk runs inside the workers.** Each worker opens the file read-only, builds a tiffgraph reader over positional reads on its own descriptor, walks it under the grammars from strictest to loosest, closes it, and passes the packet's position to the Rust identification. Nothing non-portable crosses domains. The C stubs release the OCaml runtime lock while Rust hashes and while the walker reads, so the domains work in parallel under stock OCaml and under OxCaml alike.
- **Work is handed out in batches.** phindex-ox takes `--batch` files at a time (by default eight per worker) and passes them to the pool, which spawns its domains for that batch; each domain takes the next file from the atomic counter until the batch is done, and the results come back in input order. The writer then writes the batch, commits once 500 or more results are pending, and checks whether a progress line is due. The batch size trades the cost of spawning domains against the time a slow file can hold a batch open.

Rust allocations inside phidentify's C library use mimalloc; the OCaml heap is the OCaml runtime's own.

The comparison

The two indexers were run on the same folder of 1,061 files, 4.6 GB, with 16 workers and a warm file cache, each into its own new index. phindex-ox was built in an OxCaml switch, so it ran the `Domain.Safe.spawn` pool.

- **The results were identical.** Every row of `files` and `dirs` matched in every column, error reasons included. The decompressed metadata records differed in one member only, the file access time, which changes when a file is read and which the metadata digest leaves out.

- **phindex took 13 seconds and 2.3 GB of memory.** phindex-ox took 25 seconds and 3.5 GB.

Equal results are the point: the identities do not depend on which runtime asked for them. One case is known to be open and untested: a TIFF or DNG over 1 GiB. phidentify's Rust core holds files up to 1 GiB in memory, so for a larger file phindex reports `file over the in-memory limit` in `dngsem_err`, while phindex-ox's walk reads through its own file descriptor and may accept the file and record a profile. The comparison folder held no such file, so on those files the two indexes may disagree in `dngsem_profile`, `dngsem_err` and the `preserved` columns. The comparison measured totals and did not attribute the difference in time and memory. Two structural differences are the obvious candidates: phindex streams results from a pool that never stops, while phindex-ox spawns domains per batch and waits for each batch's slowest file before the next begins. No stock OCaml timing was taken, so the comparison says nothing about stock OCaml's speed relative to OxCaml's.

For production runs, use `phindex`. Use `phindex-ox` when you want the indexer in OCaml, and as a cross-check: run both on a sample, then compare the two databases column by column.

Performance and tuning on hard disks

The first run of a large archive reads every byte, and on an array of hard disks that can take days. Whether more workers make it faster depends on which of two things limits it at the moment: the CPU or the disks.

Where the time goes

For most files, identification costs about one pass over the bytes: a SHA-256, a TIFF parse, a hash of the raw blocks, a metadata extraction, and the preserved digest. Those are fast and the disks usually limit them.

The exception is the Lightroom fingerprint. To compute it, phidentify fully decodes a DNG's raw image, which takes about 0.85 s of CPU per DNG for typical camera files, far more than reading the file. An archive of DNGs is therefore **CPU-bound** by that decode: with N cores busy, it identifies at most about N / 0.85 DNGs per second, whatever the disks could deliver. A folder of JPEGs, sidecars or non-DNG raw files has no decode and is limited by the disks.

When workers help

While the CPU limits, more workers help, up to the number of cores. The sign is that the CPU is busy across all cores, the MB/s on the progress line rises when you add workers, and the disks are not at their limit. A folder of DNGs, where the Lightroom decode dominates, is the usual case.

When the disks limit

Once the disks limit, more workers stop helping and can hurt: every worker reads a whole file, and on hard disks several concurrent readers make the heads move between files. Watch two numbers with a disk monitor such as `iostat`:

- **the read rate**, in MB/s per disk and in total. If it stays flat as you add workers, the disks are the limit.
- **the request size**, the average kilobytes per transfer. Large requests mean long sequential reads, which hard disks do well. If the request size falls as workers rise, the readers are interleaving and the disks are seeking.

On hard-disk arrays, start with as many workers as cores, compare the read rate and request size over a few minutes against a run with fewer workers, and keep the smaller number that gives the same files per second. The progress line's MB/s is the indexer's own view of the same thing.

Small files lower MB/s, not files per second

An archive is rarely all large DNGs. Lightroom's `.xmp` sidecars, Capture One's `.cop` and `.cof` settings files, thumbnails and other sidecars are small, and many of them sit beside each photo. Each costs an open, a stat, a read and a metadata extraction, and almost no bytes. In a folder full of them, the MB/s on the progress line drops sharply while the files per second barely change. That is not a slowdown, and adding workers will not bring the MB/s back. Judge progress on such folders by files per second, and expect the ETA, which is computed from bytes, to be optimistic while the indexer is among small files and pessimistic among large ones.

Memory

Each worker holds a whole file in memory while identifying it (files up to 1 GiB; larger ones are hashed by streaming), and the Lightroom decode holds the decoded raw image as well, two bytes per sample. Peak memory therefore grows with the number of workers and the size of the largest files in flight. In the comparison above, 16 workers on that folder peaked at 2.3 GB for phindex and 3.5 GB for phindex-ox.

The other memory question is the allocator. Metadata extraction through exiftool-rs makes many short-lived allocations of mixed sizes. Under the macOS system allocator, that pattern fragmented the heap in long multi-threaded runs and kept about 0.5 MB resident per extracted file, without bound. With mimalloc, memory stays flat. So phidentify's C library uses mimalloc for every Rust allocation, which covers phindex-ox, and the phindex binary sets mimalloc as its own global allocator. If you embed phidentify's Rust crate in another program, choose the allocator there; the crate leaves that choice to the program.

Queries

Open the index with `sqlite3 archive.sqlite`, or read-only from any language. The examples use generic roots; replace them with your stored, canonical paths.

Paths under a root

To select paths under a directory, use a range rather than `LIKE`, as phindex does internally. Every path under `/photos` sorts between `/photos/` and `/photos0`, because `0` is the character after `/`, and the range uses the primary-key index:

```
SELECT count(*), sum(size) FROM files
WHERE path >= '/photos/' AND path < '/photos0' AND gone = 0;
```

Containment between two roots

Which files under one root have no byte-identical copy under another?

```
SELECT a.path FROM files a
WHERE a.path >= '/photos/' AND a.path < '/photos0' AND a.gone = 0
  AND a.sha256 IS NOT NULL
  AND NOT EXISTS (
    SELECT 1 FROM files b
    WHERE b.sha256 = a.sha256 AND b.gone = 0
      AND b.path >= '/backup/' AND b.path < '/backup0');
```

The `a.sha256 IS NOT NULL` line keeps files that were never identified out of the answer; list them separately, since the index does not know yet whether they are backed up. Replace `sha256` with `dng_blocks` to ask the weaker question "is the raw data of every DNG somewhere in the backup, whatever happened to its metadata?"

Metadata-only differences

Which DNGs differ from another file only in their XMP packet? Two files with equal `preserved` digests under the same `dngsem_profile` have every byte outside the packet equal; if their SHA-256s differ, the difference is inside the packet.

```
SELECT a.path, b.path FROM files a JOIN files b
  ON a.preserved = b.preserved AND a.dngsem_profile = b.dngsem_profile
WHERE a.sha256 <> b.sha256 AND a.gone = 0 AND b.gone = 0 AND a.path < b.path;
```

Compare `preserved` only under the same profile: the profile is part of what the digest means. To see what changed in the packet, decompress the two `metadata` records and compare their XMP members, or ask whether `md_digest` differs at all. Two files whose `dng_blocks` agree but whose `preserved` digests differ have the same raw data and some other difference outside the packet, such as a new preview or a moved layout; that is not a metadata-only edit.

Lightroom catalog matching

Lightroom Classic stores a fingerprint of each original in its catalog, in the `digest` column of the `AgLibraryFileDigest` table, with the file size in `fileSize`. `lr_digest` is the same value, in the same uppercase hex, for the DNGs phidentify supports. Attach a **copy** of the catalog, never the live file:

```
ATTACH 'file:catalog-copy.lrcat?mode=ro&immutable=1' AS cat;
SELECT f.path, d.digest FROM files f
JOIN cat.AgLibraryFileDigest d
  ON d.digest = f.lr_digest AND d.fileSize = f.size
WHERE f.gone = 0;
```

Opening the copy with `immutable=1` keeps SQLite from creating journal files beside it; the `file:` form needs URI filenames, which the `sqlite3` shell accepts. An equal fingerprint and size means Lightroom Classic's matcher would accept the file as that original. It does not mean equal bytes or equal pixels. A file with `lr_err` set gets no answer from the index, not a "no": the fingerprint is defined only for the supported strata. Join `AgLibraryFileDigest` to the catalog's other tables to see which catalog entry each digest belongs to.

Housekeeping

```
-- why identities failed, most common first
SELECT lr_err, count(*) FROM files WHERE lr_err IS NOT NULL AND gone = 0
GROUP BY lr_err ORDER BY 2 DESC;

-- files still to read (never identified, failed, or from an older revision)
SELECT count(*) FROM files WHERE gone = 0
  AND (sha256 IS NULL OR ids_rev IS NOT
       'phidentify 0.1.0 +lightroom +metadata; tiffgraph dng-xmp/0,1,2 max_xmp=1MiB;
SDE/0-preserved');

-- the last runs
SELECT root, datetime(started_unix, 'unixepoch') AS started,
       finished_unix - started_unix AS secs,
       walked, hashed, bytes_hashed, gone, errors
FROM runs ORDER BY started_unix DESC LIMIT 10;
```

Testing

The integration tests in `rust/tests/incremental.rs` run the phindex binary on disposable synthetic trees in a temporary directory, with two workers:

- **Reruns read only what changed.** A first run reads six files; an unchanged rerun reads none; a new nested folder is found by the quick walk and its one file read; a file rewritten in place is not read by the quick walk and is read by `--full-stat`, with a new SHA-256; a deleted file is marked gone and not counted again on the next run.
- **Moves keep identities, and `--only` limits the walk.** Renaming a folder of three files reads nothing and reports three moved and three gone, with the SHA-256 carried to the new path. With `--only` on a new folder, the new file is read and a file moved into the folder from outside it is carried, and its old path, outside `--only`, is marked gone. A file moved away and then back to its old path, unchanged, is not read and is no longer marked gone.
- **Identities are recorded separately.** For a text file, the SHA-256 is present, the DNG identities are absent, the metadata digest is present, the refused grammar leaves a reason in `dngsem_err`, and one metadata record is stored.

```
cd rust && cargo test --release
```

The equivalence of phindex and phindex-ox rests on comparing their databases on the same tree, as in the comparison above. Synthetic tests check the indexing logic; they do not survey real camera files, and they do not qualify the identities, which phidentify's manual covers.

Licences

The phindex source is MIT. The binaries link third-party code under other licences, through phidentify:

- **rawler**, under LGPL-2.1, decodes raw samples for the Lightroom fingerprint.
- **exiftool-rs**, under GPL-3.0-or-later, extracts the metadata record, through dng-compare's FFI crate.

Built phindex and phindex-ox binaries are therefore covered by the GPL. phindex only reads the indexed tree; it writes only its SQLite file.

Reference

Item	Value
Version	0.1.0
Revision stamp	`phidentify 0.1.0 +lightroom +metadata; tiffgraph dng-xmp/0,1,2 max_xmp=1MiB; SDE/0-preserved`
Metadata engine label	`exiftool-rs (via phidentify)`
Tables	`files`, `metadata`, `dirs`, `runs`
Commit interval	every 500 results; the plan in one transaction before reading
Journal mode	WAL
Defaults	`--workers 8`, `--progress-secs 60`, phindex-ox `--batch` = workers × 8
Errors printed	the first 50 per run
In-memory limit	1 GiB (phidentify); larger files get SHA-256 only
Long strings in records	over 1 KiB, as `long-sha256:<hex>:<length>`
Record compression	raw deflate, no zlib header
Rust	1.99.0, edition 2024; rayon pool, one SQLite writer, mimalloc
OCaml	5.1 or later: stock OCaml (`Domain.spawn`) or OxCaml (`Domain.Safe.spawn`)
Source	github.com/firstpair/phindex
Related projects	phidentify, tiffgraph, Semantic Unison, dng-compare